Projet n°5 du parcours Data Scientist Machine Learning

Titre : Déployer un modèle de machine learning

Le 28/09/2026

1. Présentations

1 La mission

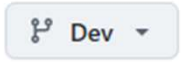
La mission consiste a déployer un modèle de machine learning à l'aide d'outils modernes, pour cela nous avons le choix de déployer soit le modèle du projet 3 (Anticipez les besoins en consommations de bâtiment) ou bien celui du projet 4 (Classifiez automatiquement des informations).
J'ai choisi de déployer le modèle du projet 4 (Classifiez automatiquement des informations).

2.gitHub

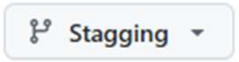
2.1 Mettez en place un système de gestion de version et collaboration

Repository distant nommé ’P5-Deployez-un-modele-de-Machine-Learning’

Trois branches Dev/Stagging/Main.



Le code du développement



Tests de l’application



Déploiement en production

Description des commit : conventionnel commit.

L’outil de résolution des conflit est celui intégré a Git, je les résout en ligne de commande ou depuis l'interface.

La gestion des version par tags est faite avec l’outil : semantic release.

La gestion des secrets : .env à ajouter par les développeur, aucun secret n’est comités.

3 Configurez la CI/CD

3.1 Des fichiers .yaml docker-compose et dockerfile

Des fichiers descriptifs .yaml pour le backend et frontend orchestré par un fichier docker-compose.

1-Le serveur backend Fast API est créer via un dockerfile.
2-Le Frontend est créer via un dockerfile
3-Le serveru de base de donnée estr créé via l'orchestrateur.

Le serveur postgres répond être disponible sur le port :5432, la base est créé mais il n'y a aucune table.
Le serveur backend FastAPI répond sur le port :8000 avec une première route / et un message succinct Uvicorn running on http://0.0.0.0:8000.

-Le serveur html JavaScript effectue des requetes fetch vers le backend, le backend interroge la base de données Via des controllers qui effectues des requêtes SQL, puis renvois les informations en frontend sous le format json.

3.2 Capture d'écran du fichier docker-compose

Concernant la gestion des secrets : **Aucun mots de passe** de la base de données ne sont trouvent écrit dans le fichiers descriptif .yml. Le .env qui injecte les mots de passe dans le fichier est ignoré lors des commit grâce au fichier .gitignore.

```yaml
services:
  postgres:
    image: postgres:15-alpine
    container_name: postgres_rh_db
    environment:
      POSTGRES_USER: ${DB_USER:-postgres}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: ${DB_NAME:-rh_predictions_db}
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d rh_predictions_db"]
      interval: 5s
      timeout: 5s
      retries: 10
```

```yaml
  api:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: fastapi_app
    depends_on:
      postgres:
        condition: service_healthy
      db-init-csv:
        condition:
service_completed_successfully
    environment:
      DB_HOST: postgres
      DB_PORT: 5432
      DB_USER: ${DB_USER:-postgres}
      DB_PASSWORD: ${DB_PASSWORD}
      DB_NAME: ${DB_NAME:-rh_predictions_db}
    ports:
      - "8000:8000"
    volumes:
      - ./backend/modeles:/app/modeles
```

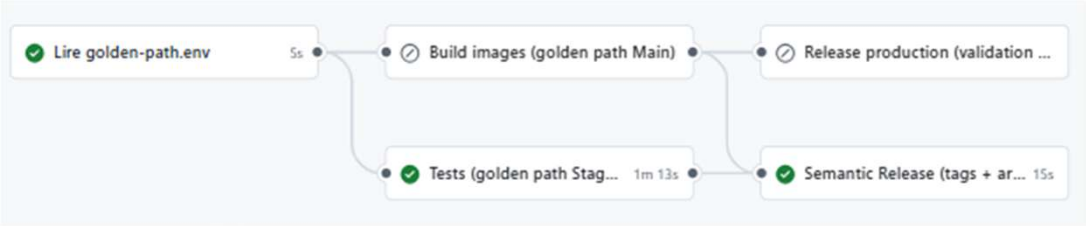
```yaml
frontend:
 build:
   context: ./frontend
   dockerfile: dockerfile
 container_name: p5_frontend
 depends_on:
   - api
 ports:
   - "8080:80"
```

	Name	Container ID	Image	Port(s)	CPU (%)
●	p5-deployez_un_modele_de_machine_learning	-	-	-	0.13%
●	p5_frontend	0d770389b2dd	p5-deployez_un_modele_de_machine_learning-frontend	8080:80	0%
●	fastapi_app	3dc2c68d62e2	p5-deployez_un_modele_de_machine_learning-api	8000:8000	0.13%
●	postgres_rh_db	2f3c3c66ab63	postgres:15-alpine	5432:5432	0%

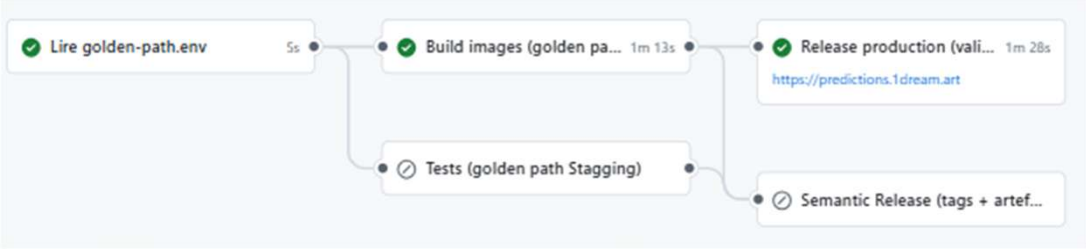
3.3 Le fichier CI-CD.yml pour gérer les gitHub actions

Les différentes github actions :

1-Si un commit ou merge request sur la branche Stagging : Le Job de test se déclenche ainsi que la création d'un **artefact avec pytest** incorporant les résultats et la couverture des tests puis **un artefact de l'application versionné est créé sur le serveur github avec semantic release** si les tests n'ont pas échoués.



2- Si un commit ou merge request sur la branche Main : Le dernier artéfact versionné est récupéré depuis la banche Stagging et suite a une review (approbation manuelle) le déploiement en production peut avoir lieu.



4. l'API REST et la base de donnée

4.1 Création des routes et des controllers pour la logique métier

Depuis le frontend :

Requêtes crud fetch http + entêtes (headers) + gardefou (la requete se coupe au bout de n signal)

Depuis le backend :

La route envoie vers le controller les paramètres envoyée par le frontend.

Le backend reçoit la requête effectue les reqeutes SQL, et renvois le résultat.

4.2 Détail des routes CRUD

Auth

GET	/auth/sante	Aucune base
POST	/auth/register	ORM UserDB
POST	/auth/login	ORM UserDB
GET	/auth/moi	Jeton seulement
DELETE	/auth/compte	ORM UserDB
POST	/auth/mot-de-passe	ORM UserDB

Prédiction

GET	/colonnes	Aucune base
POST	/predict	ORM ResultatDB

Données

POST	/sauvegarder	ORM ResultatDB
GET	/resultats	ORM ResultatDB
GET	/resultats/{id}	ORM plus SQL brut pour les features
GET	/annuaire	SQL brut
GET	/employes/{id}	SQL brut

4.2 Schéma de la base de donnée

```
rh_predictions_db-# \dt
             List of relations
 Schema |       Name        | Type  |  Owner
--------+-------------------+-------+----------
 public | employes_cible    | table | postgres
 public | employes_features | table | postgres
 public | resultats         | table | postgres
(3 rows)
```

```
rh_predictions_db-# \d resultats
                                      Tabl
        Column          |       Type        |
------------------------+-------------------+
 id                     | integer           |
 prenom                 | character varying |
 nom                    | character varying |
 modele_utilise         | character varying |
 probabilite_de_quitter | double precision  |
 prediction             | integer           |
 libelle_prediction     | character varying |
 seuil_applique         | double precision  |
 details                | character varying |
Indexes:
    "resultats_pkey" PRIMARY KEY, btree (id)
    "ix_resultats_id" btree (id)
```

```
rh_predictions_db-# \d employes_features
                                      Table "public
                Column                |       Type        |
--------------------------------------+-------------------+
 id                                   | integer           |
 age                                  | integer           |
 genre                                | character varying |
 revenu_mensuel                       | integer           |
 statut_marital                       | character varying |
 departement                          | character varying |
 poste                                | character varying |
 nombre_experiences_precedentes       | integer           |
 annee_experience_totale              | integer           |
 annees_dans_l_entreprise             | integer           |
 annees_dans_le_poste_actuel          | integer           |
 nombre_participation_pee             | integer           |
 nb_formations_suivies                | integer           |
 distance_domicile_travail            | integer           |
 niveau_education                     | integer           |
 domaine_etude                        | character varying |
 frequence_deplacement                | character varying |
 annees_depuis_la_derniere_promotion  | integer           |
 annes_sous_responsable_actuel        | integer           |
 satisfaction_employee_environnement  | integer           |
```

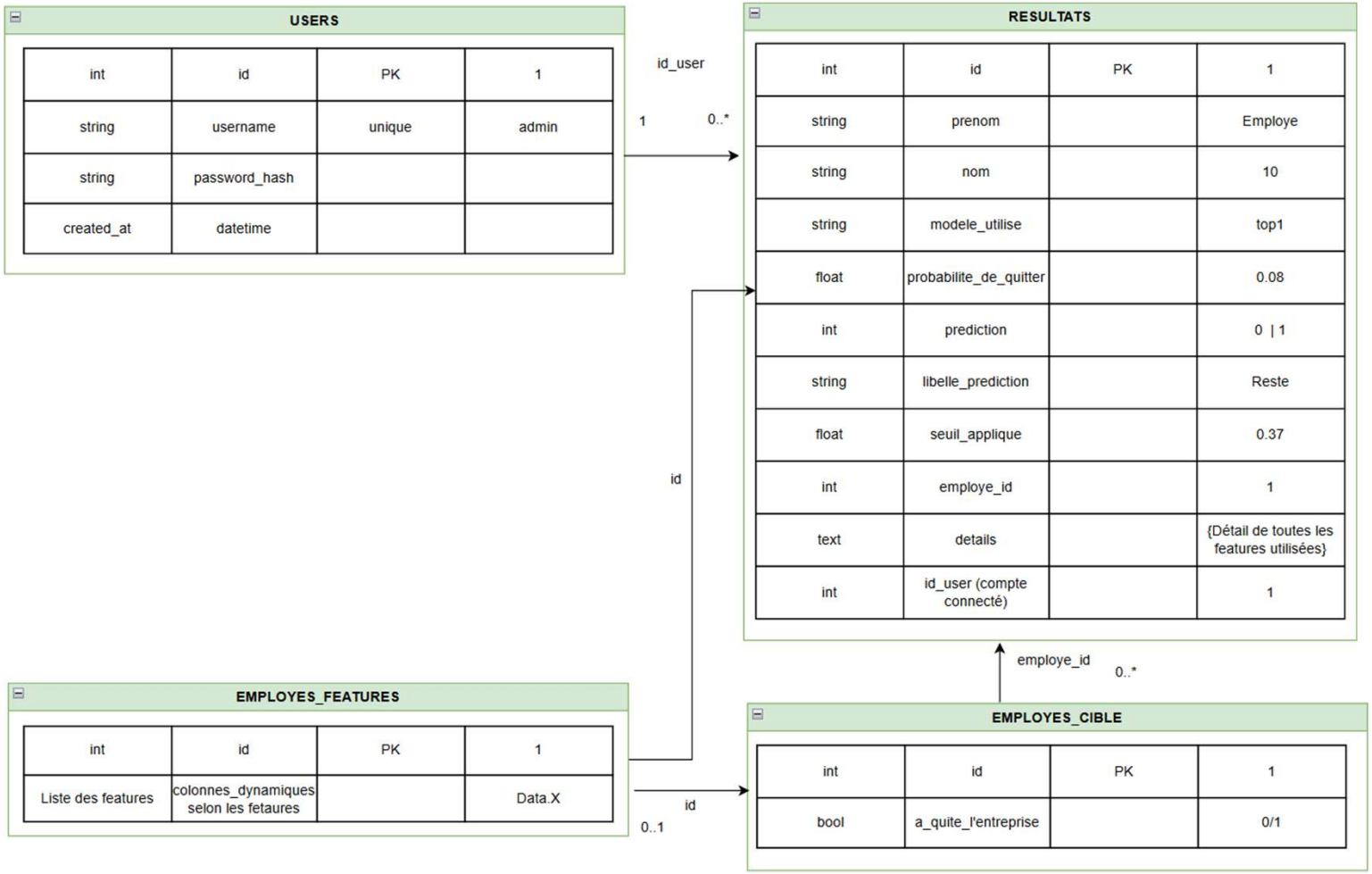
```
rh_predictions_db-# \d employes_cible
                                  Tab
       Column          |       Type        |
-----------------------+-------------------+
 id                    | integer           |
 a_quitte_l_entreprise | character varying |
```

rh_predictions_db=# SELECT * FROM resultats LIMIT 10;

 id | prenom | nom | modele_utilise | probabilite_de_quitter | prediction | libelle_prediction | seuil_applique | details
----+---------+------+----------------+------------------------+------------+--------------------+----------------+---------

 1 | John | Doe | top1 | 0.05627246977416935 | 0 | Reste | 0.37 | {"age": 36, "revenu_mensuel": 4919, "annees_dans_l_entreprise": 5, "distance_domicile_travail": 7, "annes_sous_responsable_actuel": 3, "satisfaction_employee_environnement": 3, "niveau_hierarchique_poste": 2, "satisfaction_min": 2, "poste_x_niveau": 6, "formations_par_an": 0.3, "statut_marital": "Célibataire", "frequence_deplacement": "Aucun", "heure_supplementaires": "Non", "hs_et_salaire_bas": 0, "jeune_faible_anciennete": 0, "nb_formations_suivies": 3, "annee_experience_totale": 10, "niveau_education": 3, "genre": "M", "departement": "Commercial", "poste": "Assistant de Direction", "domaine_etude": "Autre"}

4.3 Schéma de la base de donnée



4.3 Validation des entrées avec Pydantic

Les DTO (Data Transfer Object) sont appelés schémas dans FastAPI.

La validation des données est faite par Pydantic. Ils servent aussi de couche de sécurité : on reconstruit un objet contrôlé à partir du JSON reçu, et on peut exposer au frontend un contrat différent des tables en base.

Pydantic valide les payloads à l'entrée de l'API (types, champs obligatoires). Les cas limites mal formés sont rejetés avant le métier. Ça réduit la surface d'attaque, mais la sécurité repose aussi sur l'ORM paramétré, les tests d'API et l'absence d'exposition directe des tables.

Les erreurs sont ainsi renvoyer au frontend sous le format .json
Payload invalide (Pydantic) → 422.

4. L'Insérez le dataset et gérez-le via PostgreSQL

Le script insert_X_and_y_tables.py se lance au démarrage de l'orchestrateur docker-compose,
dans un conteneur appelé db-init-csv.
Il lit X.csv et y.csv, et transfert les informations de X.csv et y.csv, si elles n'existent pas en base, dans deux tables distinctes grâce à des requêtes SQL simplifiée par l'ORM.

→ Charger_donnees_si_vide : compte les lignes (SELECT COUNT(*)). Si la table n'est pas vide, pas de rechargement (évite les doublons). Si elle est vide, les lignes du CSV sont insérées par lots via un INSERT généré par SQLAlchemy Core, exécuté par le driver psycopg.

4-Gestion via l'ORM SQLAlchemy (Ecrit et lit des objets métiers crées par l'application)
/sauvegarder POST + modèle ORM
/resultats GET + modèle ORM

* SQLAlchemy génère le SQL, le driver psycopg l'envoie à Postgres (Ici containerisé).

Deux manière d'intéragir avec des données
1-Via un ORM et un modèle

```python
class UserDB(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    username = Column(String(80), unique=True, nullable=False,
index=True)
    password_hash = Column(String(255), nullable=False)
    created_at = Column(DateTime, default=datetime.utcnow,
nullable=False)
```

```python
def creer_compte(username: str, password: str) -> dict:
    identifiant = username.strip()
    db = SessionLocal()
    try:
        existant = db.query(UserDB).filter(UserDB.username ==
identifiant).first()
        if existant is not None:
            raise HTTPException(status_code=409, detail="Ce nom
d'utilisateur existe déjà.")
        utilisateur = UserDB(username=identifiant,
password_hash=hacher_mot_de_passe(password))
        db.add(utilisateur)
        db.commit()
        db.refresh(utilisateur)
        return creer_token(utilisateur.id, utilisateur.username)
    except HTTPException:
        db.rollback()
        raise
    except Exception as exc:
        db.rollback()
        raise HTTPException(status_code=500, detail=str(exc)) from
exc
    finally:
        db.close()
```

2-Via des requêtes SQL

```
def obtenir_employe(identifiant: int) -> dict:
    sql = """
        SELECT f.*, c.a_quitte_l_entreprise
        FROM employes_features f
        LEFT JOIN employes_cible c ON c.id = f.id
        WHERE f.id = :identifiant
    """
    try:
        with engine.connect() as conn:
            row = conn.execute(text(sql), {"identifiant":
identifiant}).mappings().first()
    except Exception as exc:
        raise HTTPException(status_code=500,
detail=str(exc)) from exc
    if row is None:
        raise HTTPException(status_code=404,
detail=f"Employé #{identifiant} introuvable.")
    return {"employe": dict(row)}
```

5. Développez des tests unitaires et fonctionnels

pytest .\tests\test_api.py --cov=backend--cov-report=term-missing --cov-report=html

25 passed, 0 failed and 0 skipped

tests 25 passed

Name	Stmts	Miss	Cover	Missing
backend__init__.py	0	0	100%	
backend\app.py	17	1	94%	14
backend\controllers__init__.py	2	0	100%	
backend\controllers\employe_controller.py	29	0	100%	
backend\controllers\modele_controller.py	272	0	100%	
backend\controllers\resultat_controller.py	79	0	100%	
backend\controllers\root_controller.py	7	0	100%	
backend\database.py	54	0	100%	
backend\models__init__.py	2	0	100%	
backend\models\resultat.py	14	0	100%	
backend\routes__init__.py	10	0	100%	
backend\routes\debug.py	6	0	100%	
backend\routes\prediction.py	14	0	100%	
backend\routes\resultats.py	20	0	100%	
backend\routes\root.py	6	0	100%	
backend\schemas__init__.py	2	0	100%	
backend\schemas\prediction.py	19	0	100%	
TOTAL	553	1	99%	

```
_____________ coverage: platform linux, python 3.13.15-final-0 _____________

Name                                          Stmts   Miss  Cover   Missing
---------------------------------------------------------------------------
backend/__init__.py                               0      0   100%
backend/app.py                                   17      1    94%   14
backend/controllers/__init__.py                   2      0   100%
backend/controllers/employe_controller.py        29      0   100%
backend/controllers/modele_controller.py        272      0   100%
backend/controllers/resultat_controller.py       79      0   100%
backend/controllers/root_controller.py            7      0   100%
backend/database.py                              54      0   100%
backend/models/__init__.py                        2      0   100%
backend/models/resultat.py                       14      0   100%
backend/routes/__init__.py                       10      0   100%
backend/routes/debug.py                           6      0   100%
backend/routes/prediction.py                     14      0   100%
backend/routes/resultats.py                      20      0   100%
backend/routes/root.py                            6      0   100%
backend/schemas/__init__.py                       2      0   100%
backend/schemas/prediction.py                    19      0   100%
---------------------------------------------------------------------------
TOTAL                                           553      1    99%
======================== 25 passed, 18 warnings in 2.13s ========================
```

6. Documentez le modèle de machine learning

